



Carátula para entrega de prácticas

Facultad de Ingeniería

Laboratorios de docencia

Laboratorio de Computación Salas A y B

Profesor(a): Ariel Adara Mercado Martinez

Estructuras de datos y Algoritmos I

Asignatura:

Grupo: 5

No de Práctica(s):

Integrante(s): González Mateo Carlos

No. de lista o brigada: 14

Semestre: 2026-2

Fecha de entrega: 17-05-2026

Observaciones:

CALIFICACIÓN: _____

Índice

Introducción.....	3
Estructuras de Datos Implementadas.....	3
Pila Dinámica (Stack).....	3
Cola Dinámica (Queue).....	3
Administración de Memoria Dinámica.....	4
Módulo Analizador (Parser) y Procesamiento de Archivos.....	4
Lectura y Mapeo de Variables (cargarArchivo).....	4
Filtrado.....	4
Algoritmo de Conversión Infija a Postfija.....	5
Jerarquía y Asociatividad de Operadores.....	5
Evaluación Matemática Postfija.....	5
Casos de Prueba y Resultados.....	6
Dificultades encontradas.....	7
Conclusiones.....	7

Introducción

Este proyecto aborda la conversión y evaluación de expresiones matemáticas en sistemas computacionales. Las expresiones escritas en notación infija, como $a + b * c$, corresponden a la forma tradicional utilizada por los humanos; sin embargo, su procesamiento requiere considerar reglas de precedencia, asociatividad y uso de paréntesis, lo que complica su evaluación directa en una computadora. Para resolver esta situación, se implementó un sistema de conversión en lenguaje C capaz de convertir expresiones infijas a notación postfija (RPN).

Estructuras de Datos Implementadas

Todas las estructuras se desarrollaron desde cero utilizando nodos enlazados y memoria dinámica, con el fin de evitar las limitaciones de los arreglos estáticos.

Pila Dinámica (Stack)

La pila funciona bajo el principio LIFO (*Last In, First Out*), donde el último elemento que entra es el primero en salir.

Primero se construyó con nodos enlazados de forma dinámica. Cada nodo contiene un puntero genérico (void*) para almacenar cualquier tipo de dato y un apuntador al siguiente nodo.

Luego se implementaron las funciones estándar: push() (insertar en la cima), pop() (extraer el elemento superior), peek() (consultar el tope sin eliminar).

Además la pila tiene dos roles importantes. Durante la conversión (algoritmo *Shunting Yard*), guarda temporalmente operadores y paréntesis y en la evaluación, almacena los operandos y los resultados parciales.

Cola Dinámica (Queue)

La cola sigue el principio FIFO (*First In, First Out*), donde el primer elemento en entrar es el primero en salir. Para poder manejarla de forma eficiente en el programa, se apoya en una lista enlazada que conserva referencias al inicio y al final, lo que facilita agregar nuevos elementos y retirar los existentes sin necesidad de recorrer toda la estructura.

Para su funcionamiento, se emplean funciones como enqueue(), que permite agregar elementos al final de la cola, y dequeue(), que retira el elemento ubicado al frente.

Dentro del sistema desarrollado, esta cola cumple un papel importante como un buffer ordenado, ya que almacena los tokens generados durante la conversión de la expresión a notación postfija, permitiendo que posteriormente puedan ser evaluados en el mismo orden en que fueron producidos.

Administración de Memoria Dinámica

A través de peticiones gestionadas con *malloc()*, el programa obtiene espacio en el *heap* en función de la magnitud de los datos procesados, garantizando una expansión dinámica. Una vez finalizado el cálculo, se ejecutan procedimientos con *free()* para desasignar ordenadamente los nodos y prevenir cualquier pérdida de recursos (*memory leaks*).

Módulo Analizador (Parser) y Procesamiento de Archivos

El módulo analizador (parser) es el encargado de leer la información de entrada y transformarla en datos que el programa pueda procesar correctamente durante la conversión y evaluación de expresiones matemáticas. Su funcionamiento se divide en dos partes principales: la lectura de archivos y la tokenización de la expresión.

Lectura y Mapeo de Variables (*cargarArchivo*)

Para permitir el manejo de variables con valores asignados dinámicamente, el sistema implementa una rutina de lectura de archivos utilizando punteros de tipo *FILE**. El archivo de entrada se procesa línea por línea mediante la función *fgets()*.

Durante la lectura, el parser verifica si la línea contiene el símbolo = usando *strchr()*. Si encuentra este carácter, interpreta que la línea corresponde a una asignación de variable. Después, mediante *sscanf(línea, "%c = %f")*, extrae tanto el identificador de la variable como su valor numérico de tipo *float*, almacenándolos en una estructura de control para su uso posterior.

Si la línea no contiene el signo =, el sistema asume que se trata de la expresión matemática principal. En este caso, se utiliza *strcspn()* para eliminar el salto de línea (*\n*) y guardar la expresión limpia en memoria.

Filtrado

Una vez que la expresión es leída por el programa, la función *infijaAPostfija()* empieza a recorrerla carácter por carácter para identificar qué tipo de elemento es cada símbolo dentro de la operación matemática.

Durante este proceso, primero se ignoran los espacios en blanco usando la función *isspace()*. Esto permite que el usuario pueda escribir expresiones con espacios sin afectar el funcionamiento del programa.

Después, el programa revisa si el carácter corresponde a una variable o letra mediante *isalpha()*. Cuando esto ocurre, el símbolo se manda directamente a la cola de salida porque los operandos conservan su orden en la expresión postfija.

Por otro lado, si el carácter corresponde a un operador matemático, este se identifica con ayuda de la función *esOperador()*, la cual reconoce operadores como +, -, *, / y ^.

Toda esta etapa sirve para separar correctamente cada elemento de la expresión antes de aplicar el algoritmo Shunting Yard, permitiendo que la conversión respete la precedencia y asociatividad de los operadores

Algoritmo de Conversión Infija a Postfija

La conversión se realizó con el algoritmo *Shunting Yard* (patio de maniobras) diseñado por Edsger Dijkstra. El programa va leyendo la expresión infija de izquierda a derecha, elemento por elemento, y según lo que encuentre va tomando decisiones para ir acomodando todo correctamente en la salida postfija.

Cuando se encuentra un operando o una variable, simplemente se manda a la cola de salida. Si aparece un operador, se revisa lo que hay en la pila para ver su prioridad; mientras el operador de la cima tenga mayor o igual precedencia (y considerando la asociatividad), se van sacando esos operadores y se envían a la salida, y después se coloca el nuevo operador en la pila.

Cuando es el caso de los paréntesis, si es uno izquierdo, se mete directo a la pila, pero si es uno derecho, entonces se van sacando operadores y mandándolos a la salida hasta encontrar el paréntesis izquierdo correspondiente, el cual se elimina junto con el derecho.

Jerarquía y Asociatividad de Operadores

Para respetar las reglas aritméticas, se definieron los siguientes niveles:

- **Nivel 3 (Alta prioridad):** $\wedge$ (potencia). Asociatividad derecha a izquierda.
- **Nivel 2 (Prioridad media):** $*$ y $/$ (multiplicación y división). Asociatividad izquierda a derecha.
- **Nivel 1 (Prioridad baja):** $+$ y $-$ (suma y resta). Asociatividad izquierda a derecha.

Evaluación Matemática Postfija

La evaluación de la expresión en notación postfija se hace recorriendo los elementos en orden, usando una pila como apoyo para ir guardando los valores que se van encontrando. Conforme se avanza, cuando aparece una variable, se toma su valor correspondiente de los datos de entrada y se guarda en la pila como un número. En cambio, si se encuentra un operador, entonces se sacan los dos últimos valores que se habían guardado, se realiza la operación con ellos y el resultado se vuelve a meter en la pila.

Este proceso se repite hasta que ya no quedan más elementos por revisar, y al final lo único que queda en la pila es el resultado de la expresión original.

Casos de Prueba y Resultados

Las pruebas se ejecutaron compilando con gcc en Kali Linux (WSL). Se usaron las variables: $a = 5$, $b = 3$, $c = 2$, $d = 3$.

Caso	Expresión infija	Postfija generada	Resultado
Suma simple	$a + b$	$a b +$	8.00
Precedencia básica	$a + b * c$	$a b c * +$	11.00
Control de paréntesis	$(a + b) * c$	$a b + c *$	16.00
Asociatividad potencia	$a \wedge b \wedge c$	$a b c \wedge \wedge$	3125.00
Expresión polinómica	$a * b + c / d$	$a b * c d / +$	16.00

[Tabla1. Resultados de validación del sistema]

```

1  a = 5
2  b = 3
3
4  c = 2
5
6  a^b^c
7

```

```

(Daxsse@DESKTOP-84FL2QL) - [~/EDA/PostFija2/postfija-2-tostinachos2]
$ ./programa
Expresion infija:
(a+b)*c

Expresion postfija:
a b + c *

Resultado:
16.00

(Daxsse@DESKTOP-84FL2QL) - [~/EDA/PostFija2/postfija-2-tostinachos2]
$ ./programa
Expresion infija:
a^b^c

Expresion postfija:
a b c ^ ^

Resultado:
512.00

(Daxsse@DESKTOP-84FL2QL) - [~/EDA/PostFija2/postfija-2-tostinachos2]
$ ./programa
Expresion infija:
a^b^c

Expresion postfija:
a b c ^ ^

Resultado:
1953125.00

```

En pruebas adicionales con otros valores (por ejemplo, $a = 4$, $b = 5$, $c = 2$), los resultados también fueron correctos, incluyendo la asociatividad derecha de la potencia.

[Imagen 1. Pruebas]

Dificultades encontradas

Durante el desarrollo del programa surgieron varios problemas que fue necesario resolver para que el sistema funcionara correctamente. Uno de los primeros errores fue un fallo de segmentación en la función *push()*, ya que inicialmente se intentaba copiar información con *memcpy()* sin haber reservado memoria previamente para el nuevo nodo. Esto provocaba un *Segmentation Fault*, por lo que se corrigió asegurando que antes de cualquier copia se realizara la asignación de memoria con *malloc()*.

Otro aspecto complicado fue el manejo de diferentes tipos de datos dentro de la pila. Aunque la estructura utilizaba *void** para mantener una implementación genérica, el parser trabajaba principalmente con caracteres, mientras que la evaluación matemática necesitaba valores numéricos tipo *float*. Esto generó algunos problemas de compatibilidad entre ambas etapas. La solución consistió en manejar cada caso mediante funciones específicas sin modificar completamente la estructura principal de la pila.

También se presentó un problema al momento de mostrar la expresión en notación postfija, ya que los elementos aparecían pegados entre sí (por ejemplo, *ab+c**), lo que dificultaba su lectura. Para solucionarlo, se ajustó la impresión de la cola para incluir espacios y así mejorar la claridad de la salida.

El parser se diseñó para leer y procesar expresiones de forma flexible, permitiendo distintos formatos de escritura dentro del archivo de entrada.

En la función *cargarArchivo()*, la lectura de variables se realiza utilizando *sscanf(linea, "%c = %f")*. El espacio colocado antes de *%c* permite ignorar espacios en blanco automáticamente, haciendo posible procesar expresiones como *a=5*, *a = 5* o incluso *a = 5.0* sin afectar el resultado.

Por otro lado, dentro de la función *infijaAPostfija()*, la expresión se analiza carácter por carácter. Para ello se utilizan funciones como *isspace()* e *isalpha()*, las cuales ayudan a distinguir espacios, variables y operadores. Los operandos se envían directamente a la cola de salida, mientras que los operadores se manejan mediante la pila para respetar la precedencia y asociatividad durante la conversión a notación postfija.

Conclusiones

Con el proyecto se logró entender mejor cómo funcionan las estructuras dinámicas (pero ahora en un caso más práctico y no solo en teoría). Al usar pilas y colas se me hizo más claro cómo sirven para organizar operaciones y resolver expresiones matemáticas paso a paso.

También me sirvió para practicar el manejo de memoria dinámica en C, sobre todo con punteros y la reserva manual de memoria. Los errores que fueron saliendo durante el desarrollo me ayudaron a comprender mejor cómo funciona la memoria y por qué es clave validar cada operación bien.